Meshtryoshka: Differentiable Rendering of Real-World Scenes via Mesh Rasterization

DAVID CHARATAN, Massachusetts Institute of Technology
DANIEL XU, Massachusetts Institute of Technology
RICHARD SZELISKI, University of Washington
GEORGE KOPANAS, Runway AI, Inc.
VINCENT SITZMANN, Massachusetts Institute of Technology

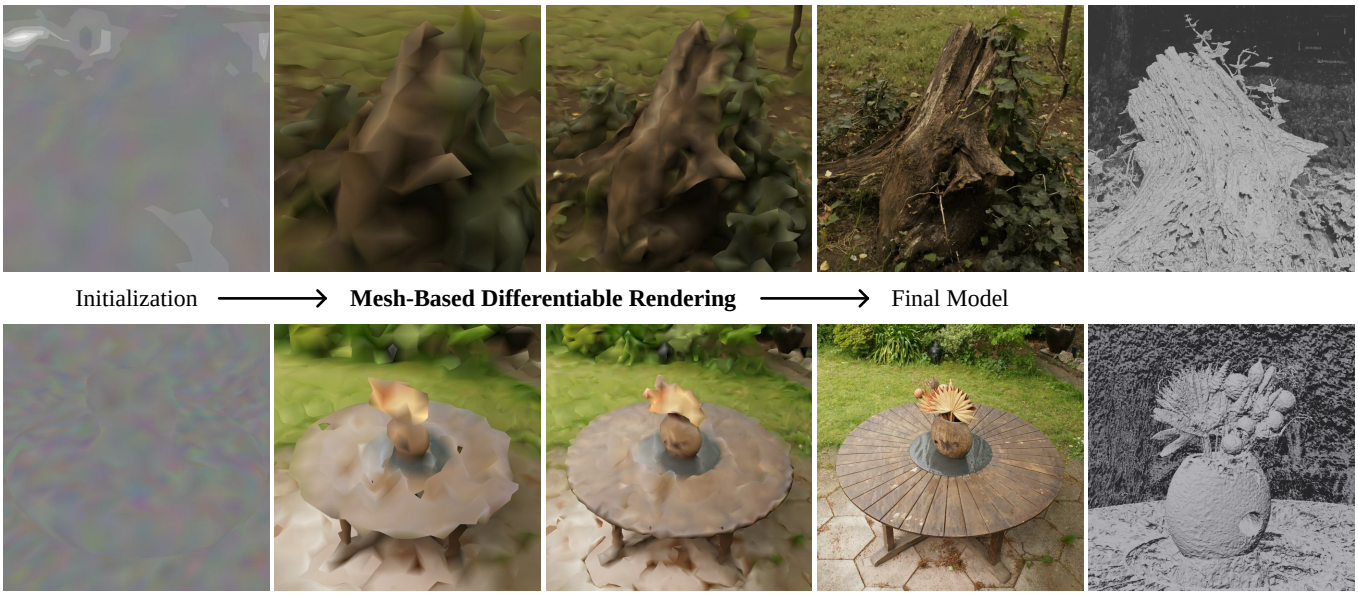


Fig. 1. We present *Meshtryoshka*, a mesh-based differentiable rendering method that is capable of reconstructing both object-centric and real-world scenes. Our method extracts multiple level sets of a signed distance function, individually renders the resulting triangle meshes using a non-differentiable rasterizer, and then alpha-composites the resulting images to produce a rendered view. Unlike previous mesh-based differentiable renderers, our method is able to reconstruct real-world, unbounded scenes.

Differentiable rendering has emerged as a powerful approach for 3D reconstruction and novel view synthesis. Today, state-of-the-art differentiable rendering methods combine a variety of custom representations of 3D geometry and appearance with specialized renderers. However, most downstream tasks in computer graphics rely on 3D meshes, which provide superior portability, allow for hardware-accelerated rendering, and are at the core of most computer graphics workflows. While prior work has attempted differentiable rendering with mesh representations, these approaches are limited to object-centric scenes and fail to reconstruct large-scale, unbounded scenes. In this work, we introduce *Meshtryoshka*, a novel mesh differentiable rendering framework that combines an off-the-shelf triangle rasterizer with a 3D representation that consists of nested mesh shells which resemble a matryoshka doll. In every forward pass, the mesh shells are extracted anew from a 3D signed distance function via iso-surface extraction, and the opacities for each vertex are computed as a function of signed distance. Each mesh shell is then rasterized independently, and the final image is created via alpha compositing. Crucially, mesh vertex positions are updated only indirectly via gradients that flow through the opacity values into the signed distance function, and hence, our method is compatible with off-the-shelf mesh renderers that need not be differentiable with respect to vertex positions. On object-centric scenes, our method performs competitively with surface-based differentiable rendering techniques. To the best of our knowledge, our method is the first differentiable mesh rendering method that scales to unbounded, real-world 3D scenes, where it yields high-quality novel view synthesis results approaching those of state-of-the-art, non-mesh methods. Our method suggests that it may be possible to solve the differentiable rendering problem without relying on specialized renderers, only using conventional tools from the computer graphics toolbox.

CCS Concepts: • **Computing methodologies** → **Rendering**.

Authors' addresses: David Charatan, Massachusetts Institute of Technology, charatan@mit.edu; Daniel Xu, Massachusetts Institute of Technology, danielxu@mit.edu; Richard Szeliski, University of Washington, szeliski@cs.washington.edu; George Kopanas, Runway AI, Inc., george@runwayml.com; Vincent Sitzmann, Massachusetts Institute of Technology.

ACM 0730-0301/2025/7-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Additional Key Words and Phrases: novel view synthesis, differentiable rendering, radiance fields, real-time rendering

ACM Reference Format:
David Charatan, Daniel Xu, Richard Szeliski, George Kopanas, and Vincent Sitzmann. 2025. Meshtryoshka: Differentiable Rendering of Real-World Scenes via Mesh Rasterization. *ACM Trans. Graph.* 1, 1 (July 2025), 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Since the dawn of computer graphics, researchers have proposed countless pairings of 3D scene representations and renderers, each offering distinct trade-offs of speed, quality, editability, and simplicity. Nevertheless, the most widely used 3D representation remains the 3D surface mesh. Meshes are portable, support fast rendering via dedicated hardware implementations, and are essential to many computer graphics workflows, including modeling, animation, and physics simulation. Yet despite the general consensus surrounding 3D surface meshes, differentiable rendering operates primarily through alternatives to traditional mesh rendering. In differentiable rendering, we render a 3D scene representation, compare it with ground-truth images to compute a loss, and backpropagate into the representation to reconstruct an underlying scene. Variants have emerged that differ in their 3D representations, parameterizations, and rendering formulations, each with pros and cons. However, there is no differentiable renderer that both achieves photorealistic results on real-world scenes and directly optimizes a mesh.

Common wisdom holds that surface meshes are difficult to optimize. To circumvent this issue, differentiable volume rendering [Mildenhall et al. 2020] and volume-surface hybrids [Wang et al. 2023a; Yariv et al. 2021] leverage volumetric density as an easy-to-optimize representation, avoiding local minima related to changing topology and transporting primitives via gradient descent. Nonetheless, some mesh-based works have achieved impressive reconstruction results by parameterizing surface meshes as the output of isosurface extraction algorithms and optimizing an underlying signed distance function [Shen et al. 2021, 2023]. Yet such methods are constrained to simple, object-centric scenes, require foreground-background masks, and fail to reconstruct unbounded, real-world 3D scenes.

In this paper, we introduce a differentiable rendering framework that enables the direct optimization of mesh representations of unbounded, real-world 3D scenes. At the core of our method are grids of signed distance values and spherical harmonics that are initially dense, but are dramatically sparsified over the course of optimization. During each optimization step, we extract a set of nested mesh shells at regularly spaced level sets via Marching Cubes; this inspires the name of our method, *Meshtryoshka*. We visualize an overview of the way we extract the mesh shells in Fig. 4. We then independently rasterize each mesh shell into image space and construct final images by alpha-compositing the shells' colors and SDF-derived opacities. Finally, we backpropagate the resulting rendering error through the alpha-compositing process.

We stress that in contrast to prior work on differentiable mesh rendering, our approach is compatible with off-the-shelf rasterizers. *We do not require the renderer to be differentiable with respect to mesh vertex positions.* Vertex locations are optimized only indirectly by backpropagating into the underlying signed distance field. This enables novel and portable differentiable pipelines that do not rely on custom CUDA implementations [Kerbl et al. 2023] or stochastic estimation of gradients [Deliot et al. 2024].

We extensively validate our proposed differentiable mesh rendering framework. On object-centric scenes, we match prior work on differentiable mesh rendering without the use of foreground masks and are competitive with a recent state-of-the-art differentiable surface rendering method, NeuS2 [Wang et al. 2023a]. We then demonstrate what is—to the best of our knowledge—the first instance of reconstructing real-world, unbounded 3D scenes with a differentiable mesh renderer. Where prior differentiable mesh renderers fail, our method succeeds in reconstructing challenging scenes with high visual quality, approaching the quality of state-of-the-art, non-mesh-based differentiable renderers despite using mesh-based optimization. Finally, by virtue of our direct optimization of a mesh, the quality achieved in the optimization stage is exactly identical to the quality of the final mesh renderings.

In addition to the main method, we contribute a number of high-performance implementations of classic computer graphics algorithms, such as a sparse Marching Cubes algorithm, a software triangle rasterizer that can render more than 100,000,000 triangles at once, and a novel spatial grid arrangement for sparse Marching Cubes, all of which are essential to scaling mesh rendering to real-world, unbounded scenes.

2 RELATED WORK

Differentiable Isosurface Extraction. Classical isosurfacing methods extract a polygonal mesh that approximates the level set of a scalar function. The most well-known one is Marching Cubes [Lorensen and Cline 1987]. Differentiable versions of Marching Cubes [Guillard et al. 2024; Liao et al. 2018; Remelli et al. 2020] allow gradients to flow from extracted mesh vertices to the parameters of an underlying surface parameterization. Other methods [Gao et al. 2020; Shen et al. 2021] draw upon Marching Tetrahedra [Doi and Koide 1991] and operate on tetrahedral grids. FlexiCubes [Shen et al. 2023] extends Dual Marching Cubes to a multiresolution grid. However, most of these methods are limited by the $O(n^3)$ scaling inherent in regular 3D grids. To overcome this limitation, we extend Marching Cubes to support sparse data structures, allowing it to scale to high resolutions without the complexity of FlexiCubes' adaptive octree [Shen et al. 2023]. Further methods build upon Dual Contouring [Ju et al. 2002] and variants of Marching Cubes [Chernyaev 1995; Lopes and Brodlie 2003] to improve isosurface extraction using learned priors.

Differentiable Mesh Rendering. Previous work has explored differentiable mesh rendering. SoftRas [Liu et al. 2019] introduced an approach where gradients flow through fuzzy triangle edges for image-based 3D reconstruction tasks. Nvdiffrast [Laine et al. 2020a] provided modular primitives for high-performance differentiable rendering, significantly improving computational efficiency and versatility. Nvdiffrec [Munkberg et al. 2022a] jointly optimizes mesh geometry, materials, and lighting from images, achieving high-quality reconstructions. Shape From Tracing [Goel et al. 2020]

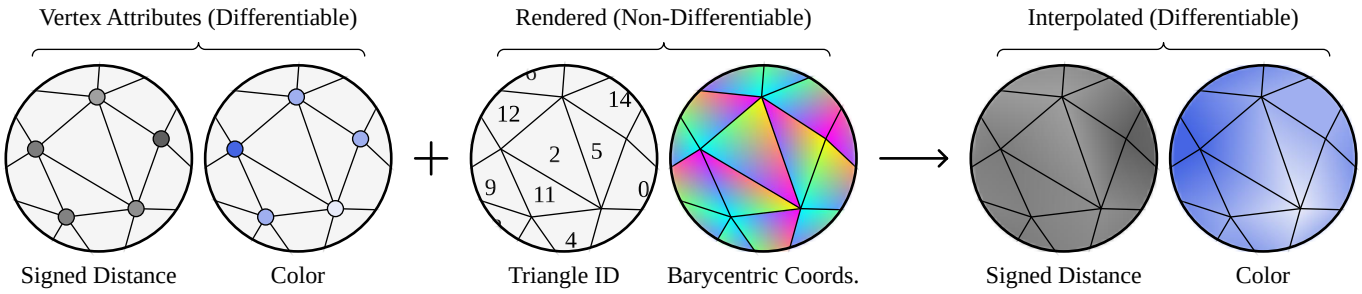


Fig. 2. The mesh extraction process (section 3.1) yields meshes with per-vertex signed distance values and spherical harmonics coefficients. We non-differentiably render these meshes to produce per-pixel triangle ID values and barycentric coordinates. After evaluating the spherical harmonics coordinates in the viewing direction to get per-vertex colors, we use differentiable interpolation (akin to a fragment shader) to yield per-pixel signed distance values and colors. These are composited using the deferred rendering process described in section 3.2 to produce a final image.

optimizes mesh geometry and materials via path tracing. More recently, DMTet [Shen et al. 2021], FlexiCubes [Shen et al. 2023], and TetWeave [Binninger et al. 2025] introduced methods combining differentiable mesh renderers with differentiable isosurface extraction, enabling robust optimization and detailed geometry reconstruction. Nicolet et al. [2021] introduces gradient preconditioning methods for faster convergence. While these methods obtain high-quality results on object-centric scenes, they generally require a foreground-background mask and do not succeed at reconstructing unbounded, real-world scenes. Furthermore, they require the mesh rasterizer to be differentiable with respect to vertex positions. Our framework matches these methods on object-centric scenes and is, to the best of our knowledge, the first mesh-based reconstruction method to reconstruct unbounded, real-world scenes.

Non-Mesh Differentiable Rendering. Differentiable rendering methods optimized for real-world, unbounded 3D reconstruction and photo-realistic novel view synthesis do not rely on meshes during optimization. Neural Radiance Fields [Mildenhall et al. 2020] instead rely on differentiable volume rendering, generally parameterized via combinations of neural fields, sparse voxel grids, and hash maps [Fridovich-Keil and Yu et al. 2022; Müller et al. 2022; Sun et al. 2022; Yu et al. 2021]. Surface-based differentiable methods generally also rely on volume rendering, but parameterize density as a function of an underlying surface representation, such as a signed distance field [Wang et al. 2021, 2023a; Yariv et al. 2021]. Gaussian Splatting [Kerbl et al. 2023] parameterizes the geometry as a set of 3D Gaussians, each with view-dependent color and opacity. This parameterization enables real-time rendering and fast optimization by leveraging the rasterization pipeline and approximations of the volume rendering integral. Contrary to our work, Gaussian Splatting requires a software rasterizer, which limits the algorithm's portability, especially during the optimization phase. To extract surfaces from Gaussian Splatting, recent methods regularize Gaussian primitives towards disks, resembling surfels [Guédon and Lepetit 2024; Huang et al. 2024; Yu et al. 2024]. These methods enable extraction of a 3D mesh in a post-processing step via iso-surface extraction. The highest quality meshes, however, are extracted via more sophisticated "baking" processes which further optimize, decimate, or regularize the extracted mesh [Chen et al. 2023; Choi et al. 2024; Esposito et al. 2024; Reiser et al. 2024; Sharma et al. 2024; Wan et al. 2023; Wang et al. 2023b; Wei et al. 2025a; Yariv et al. 2023]. While these methods enable high-quality novel view synthesis, they extract meshes a posteriori and rely on custom differentiable renderers, non-mesh 3D scene representations, and significant, multi-step processing to ultimately arrive at a 3D mesh. In contrast, we introduce a novel differentiable rendering framework that achieves competitive novel view synthesis performance by directly optimizing a 3D mesh representation via an off-the-shelf, non-differentiable 3D mesh rasterizer, suggesting that it may be possible to obtain high-quality 3D reconstruction and novel view synthesis while relying completely on the conventional computer graphics stack.

3 METHOD

We transform a standard *non-differentiable* triangle rasterizer into an effective triangle-based differentiable renderer via several key ideas. First, we parameterize scenes via grids of signed distances and view-dependent colors, allowing us to extract a set of nested mesh shells—a *Meshtryoshka*—from a scene (section 3.1). Second, we render these shells using a two-step pipeline of rasterization and deferred shading, which allows us to build upon an off-the-shelf rasterizer (section 3.2). Finally, to scale to real-world scenes, we use a mask to only store parameters where needed, efficiently subdivide the grid from coarse to fine, and stretch the grid to efficiently allocate resolution among the center and background of our scene (Section 3.3, 3.4). We additionally introduce regularizers to promote sparsity and well-behaved optimization (section 3.5). We stress that unlike previous approaches [Kerbl et al. 2023; Laine et al. 2020b], ours does not require a custom-built differentiable rasterizer.

3.1 Scene Parameterization and Mesh Extraction

We represent the scene's geometry with a signed distance function f_{SDF}. We do not define this function over the whole domain; instead, we store explicit SDF parameters θ_{SDF} at a set of point locations. One can imagine these points as a subset of a 3D grid, though in practice they are positioned differently (see section 3.4).

During each optimization step, we pass θ_{SDF} to a differentiable Marching Cubes implementation to extract multiple level sets of

f_{SDF}, each forming a separate triangle mesh. Given an iso-level ℓ, Marching Cubes creates a triangle vertex wherever neighboring signed distance values lie on opposite sides of ℓ. The vertex position is specified via an interpolation weight t, where $d_i < \ell < d_j$ are the adjacent distance values:

$$t = \frac{d_j - \ell}{d_j - d_i} \tag{1}$$

We use this interpolation weight to assign a signed distance value d_{vertex} to the triangle vertex:

$$d_{\text{vertex}} = t d_i + (1 - t) d_j \tag{2}$$

While the resulting value of d_{vertex} is trivially equal to ℓ, this equation is used to compute gradients in the backward pass. We use an additional set of parameters θ_{color}, representing view-dependent color, to store spherical harmonics coefficients for each value in θ_{SDF}. We then interpolate values $\mathbf{k}_i$ and $\mathbf{k}_j$ from θ_{color} in the same way as the SDF values to compute spherical harmonics for each triangle vertex:

$$\mathbf{k}_{\text{vertex}} = t\mathbf{k}_i + (1 - t)\mathbf{k}_j \tag{3}$$

We perform this process on multiple level sets, yielding a collection of nested triangle mesh shells—a *Meshtryoshka*—whose vertices have signed distance values d_{vertex} and spherical harmonics coefficients $\mathbf{k}_{\text{vertex}}$.

3.2 Non-Differentiable Rasterization & Deferred Rendering

In the previous section, we discussed extracting mesh shells with per-vertex signed distances and spherical harmonics coefficients. It would be possible to render these shells via ray-tracing, following the formulation from NeuS [Wang et al. 2021]. If we followed this approach, we would first compute transmittance values at each ray-triangle intersection:

$$T_i = \frac{1}{1 + \exp(-d_i)}, \tag{4}$$

Then, we would convert the transmittance values to alpha values:

$$\alpha_i = \begin{cases} 1 - T_i & \text{if } i = 1 \\ \frac{T_{i-1} - T_i}{T_{i-1}} & \text{otherwise.} \end{cases} \tag{5}$$

Finally, we would alpha-composite the values of α_i together with the corresponding colors (obtained from evaluating the spherical harmonics). However, this would require us to implement a custom ray-tracer, preventing us from using a non-differentiable, off-the-shelf rasterizer.

We will now discuss an alternative approach, based on deferred rendering, that will allow us to use a non-differentiable triangle rasterizer while still backpropagating gradients into θ_{SDF} and θ_{color}. We observe that because our method creates closely spaced, non-intersecting mesh shells, camera rays will generally intersect the shells sequentially from outermost to innermost. If the innermost shell's transmittance is chosen to be near zero (i.e., corresponding to a fully opaque surface), then it will fully occlude all further ray-shell intersections, and these further intersections will not contribute to the output image. Consequently, for each shell, we only need to take the first ray intersection into account. We thus select the values of T_i as hyperparameters such that the innermost shell's transmittance is near zero and the remaining transmittances are evenly spaced (see Figure 3). Note that by inverting equation 4, we can deduce the level sets $\ell_i = d_i$ that correspond to our chosen values of T_i. We use these for the mesh extraction described described in section 3.1.

Now, observe that given a single shell, an off-the-shelf rasterizer returns exactly what we need—the first ray-shell intersection. Specifically, at each pixel, it returns the triangle index and the barycentric coordinates (location within the triangle) of the corresponding ray-triangle intersection. While this operation is non-differentiable, it can be combined with differentiable image-space interpolation to convert per-triangle-vertex values into differentiable per-pixel values. See Figure 2 for an illustration of this process. Thus, via differentiable interpolation, we can obtain the same per-pixel signed distances and colors that a custom ray-tracer would produce.

We therefore independently rasterize each shell and interpolate the results to produce per-pixel colors and signed distances. Next, we use equations 4 and 5 to produce per-pixel alpha values. Finally, we treat the resulting images as transparent layers, alpha-compositing them to produce a final image. This allows us to backpropagate into θ_{SDF} and θ_{color} despite using a non-differentiable rasterizer.

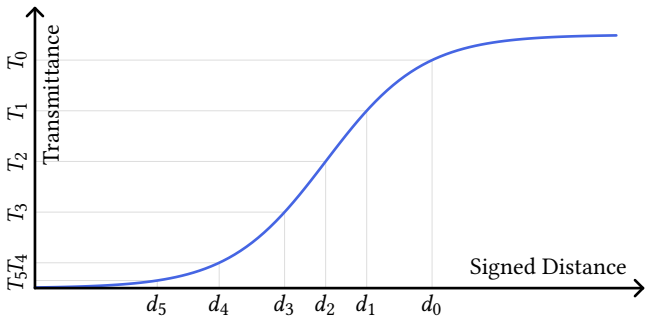


Fig. 3. We select the mesh shells' transmittance parameters T_i as hyperparameters. Then, by inverting equation 4, we deduce the corresponding level sets for iso-surface extraction. The final T_i is chosen to have a near-zero transmittance, as this allows us to approximate NeuS-style volume rendering using a single intersection per shell (see section 3.2).

3.3 Sparsity and Coarse-To-Fine Optimization

For our method, a high grid resolution is critical to representing fine geometry and textural detail. However, storing one signed distance value and $3(d+1)^2$ spherical harmonics coefficients for each grid vertex (where d is the spherical harmonics degree) would require an infeasible amount of memory. For example, assuming $d = 2$, a dense 1024^3 grid would require 120 GB of parameters.

Fortunately, voxels that correspond to empty space or solid object interiors do not contribute to the extracted shells, and so it is unnecessary to store parameters at these voxels' corners. We thus formulate our method around a sparse set of parameters and voxels and adapt Marching Cubes to this setting.

3.3.1 Voxel Extraction and Representation. Our sparse scene representation hinges upon a dense 3D grid of binary values which we refer to as the *active grid*. While this grid also suffers from cubic scaling, its memory consumption is manageable due to the fact that

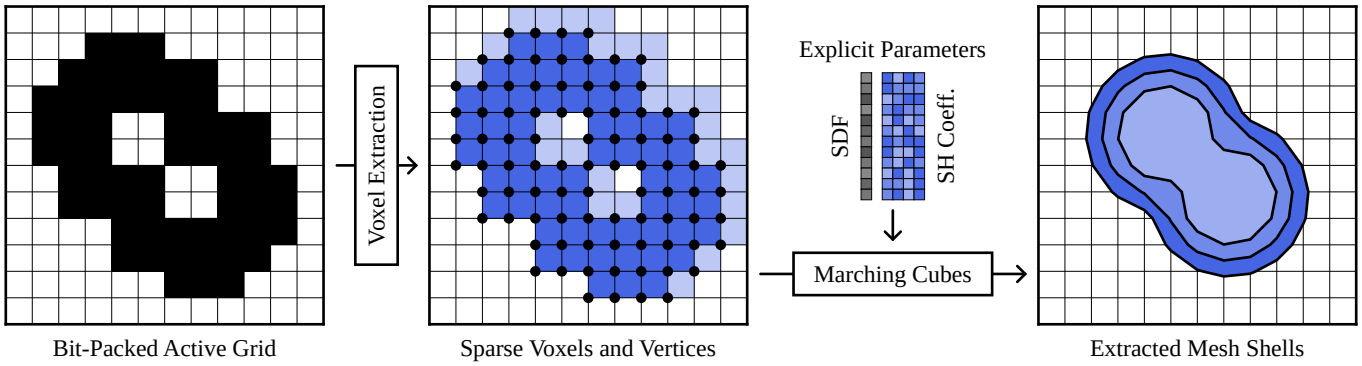


Fig. 4. An overview of the mesh extraction process. Whenever the active grid changes (i.e., after subdivision), the arrays of sparse corner vertices (**black**), active voxels (**blue**), and helper voxels (light blue) are updated. These arrays are combined with per-vertex signed distances and spherical harmonics coefficients to produce mesh shells (i.e., a Meshtryoshka) via a sparse, differentiable implementation of Marching Cubes.

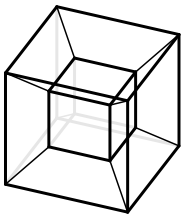
each of its values only requires a single bit. The active grid indicates which voxels Marching Cubes should run on. From this grid, we extract flat lists of active vertices and voxels. A vertex is considered active if it lies on the corner of any active voxel. Active voxels are represented via flat arrays of metadata: one for the indices of their lower corners (origin, +x, +y, +z), one for upper corners (+xy, +yz, +xz, +xyz), and one for voxel neighbor indices. The neighbor array stores neighbors in the +x, +y, +z, +xy, +yz, +xz, and +xyz directions for each voxel. In our implementation, each active voxel can be thought of as "owning" the vertex which lies at the corner with the lowest coordinate values in each dimension. As a result, there exist non-active voxels which "own" corners. We refer to these as *helper voxels* and represent them as additional values at the end of the flat array of lower corners. Unlike active voxels, helper voxels do not have upper corners or neighbors. We update the lists of vertices, lower corners, upper corners, and neighbors every time the active grid changes (see section 3.3.3). Figure 4 shows an overview of this process.

3.3.2 Sparse Marching Cubes. Given the above data structures and a list of per-vertex model parameters, our method follows classical Marching Cubes to extract triangles within each active voxel. This occurs in two steps: triangle vertex creation and triangle face creation. The vertex creation step is parallelized over all active voxels and helper voxels; it creates a triangle vertex on a voxel edge whenever the signed distance values at the edge's endpoints are on opposite sides of the level set. Note that each voxel only creates triangle vertices on edges adjacent to its lowest coordinate, meaning that this step exclusively relies on the lower neighbors and can hence be applied to both active and helper voxels. On the other hand, the face creation step is parallelized only over active voxels and additionally requires the neighbor array as input. In this step, we use the neighbor array to deduce the indices of the triangle vertices created by adjacent active and helper voxels.

3.3.3 Subdivision. At initialization, our optimization pipeline begins with a fully occupied low-resolution active grid. At a fixed interval of optimization steps, we prune and subdivide the grid. To prune the grid, we extract the zero level set of the current scene and mark all voxels that contribute to the zero level set as being active. We then dilate and upscale the resulting active grid. After extracting a new set of vertices for the upscaled active grid (see section 3.3.1), we trilinearly interpolate the previous subdivision level's signed distance and spherical harmonics values. Because the trilinear interpolation here closely mirrors the one used in Marching Cubes, our subdivision operation only minimally affects the scene's appearance and does not interrupt the optimization process.

3.4 Non-Cubic Grids for Real-World Scenes

When fitting our model to object-centric scenes, we use a regular 3D grid in conjunction with the sparse representation described above. However, real-world scenes cover much greater spatial extents, and so even a sparse regular grid would be too memory-inefficient for our use case. We therefore divide real-world scenes into foreground and background regions and handle each one separately. In general, the foreground region is defined as the axis-aligned cube that bounds the cameras, while the background region is defined as all other space. We use a regular 3D grid to model the foreground region and represent the background region using six truncated frustums (see inset).



Each of these frustums contains frustum-shaped voxels that are compatible with the standard Marching Cubes formulation. To ensure that these voxels are roughly the same size along each dimension, we apply exponential scaling to the voxel corners' distance from the origin. See Figure 5 for a visualization of this process. Note that our approach to positioning grid vertices has a similar motivation to the spatial warping and contraction functions from existing NeRF-like methods [Barron et al. 2022; Neff et al. 2021; Reiser et al. 2023; Zhang et al. 2020]. However, as Figure 5 shows, our strategy is particularly suited to our grid-based approach.

3.5 Regularization

SDF-based methods often use an Eikonal loss which encourages the SDF gradient magnitude to be 1 everywhere. However, applying an Eikonal regularizer to explicit grid parameters tends to create

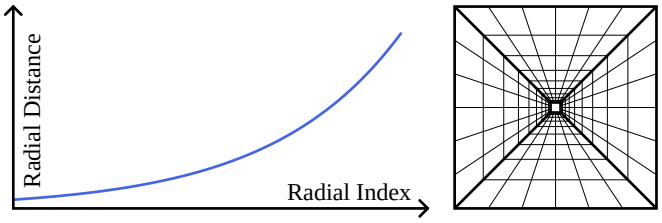


Fig. 5. A visualization of the frustum vertices' spatial distribution. As shown on the left, the vertices' distance from the center increases exponentially. This allows the model to allocate fewer parameters to far-away parts of the scene. The exponential vertex placement also ensures that the voxel dimensions (width and height in the 2D illustration on the right) grow in equal proportion to each other.

zig-zag or checkerboard patterns where each voxel locally satisfies the Eikonal constraint, but the overall signed distance field is unusable. To circumvent this issue, we instead apply a Laplacian smoothness loss, encouraging each SDF value to match the average of its neighbors. This promotes smoothness while implicitly regularizing normals and discouraging high-frequency noise. To prune geometry in unobserved regions, we reward SDF values with high L1 norm, encouraging the values to become strongly positive or negative. Note that as a result of our choice of regularizers, our signed distance parameters do not form a valid signed distance field—they do not respect the Eikonal constraint. However, we have presented our method in terms of signed distance fields since the reader is likely to be familiar with them and our method behaves similarly to existing SDF-based methods in practice. We observe that spherical harmonics cause the model to fall into local minima, where the model can cheat by misrepresenting the geometry to different views. To limit this effect, we regularize by lowering the learning rate of the non-DC components of the spherical harmonic parameters by 20x and adding an L2 penalty.

3.6 Implementation

Our model is primarily implemented using PyTorch [Paszke et al. 2019], with performance-critical components written using the Slang shader language [Bangaru et al. 2023]. We use a Slang-based triangle rasterizer because the preeminent alternative, nvdiffrast [Laine et al. 2020b], only supports up to 16,777,216 triangles. Our implementation of Marching Cubes is based on the DISO package [Wei et al. 2025b], but has been ported to Slang and modified to support sparse inputs. We use 5 mesh shells that correspond to transmittance values of 0.9, 0.5, 0.1, 0.01, and 0.001. For object-centric scenes, we initialize the signed distance function as a sphere. For unbounded scenes, we follow Gaussian Splatting [Kerbl et al. 2023] and initialize θ_{SDF} from an initial point cloud, placing a small sphere around each point.

4 RESULTS AND EVALUATION

We evaluate our model's performance on the NeRF-Synthetic and Mip-NeRF 360 datasets, which we use as representative collections of object-centric and real-world scenes. We report PSNR, SSIM [Wang et al. 2004], and LPIPS [Zhang et al. 2018] on held-out test views.

4.1 Baselines

We compare our method to mesh and non-mesh-based differentiable rendering approaches. On object-centric scenes, we primarily compare our method to nvdiffrec [Munkberg et al. 2022b], which combines differentiable Dual Marching Cubes with an edge-gradient-based triangle rasterizer. Beyond mesh optimization, we compare our method to representative methods from other classes of differentiable renderers: Zip-NeRF for volumetric approaches and 3D Gaussian Splatting for primitive-based ones. Finally, we compare our approach to Volumetric Surfaces, which bears some similarity to our method because because it bakes a NeuS-like model into multiple semi-transparent mesh shells.

4.2 NeRF-Synthetic

To evaluate our method on object-centric scenes, we conduct experiments on the NeRF-Synthetic dataset. Our model is initialized as a sphere at a resolution of 64 and progressively subdivided every 800 training steps, reaching a final resolution of 1024. We report qualitative results in the second half-page of Figure 8 and quantitative results in Table 1. Please find further qualitative video results in the Supplemental Material. Qualitatively, our method succeeds at capturing fine geometric detail. On the LEGO scene, our method resolves minute holes in the bulldozer's tracks. Our method outperforms nvdiffrec+DMTet, better capturing fine detail, and slightly outperforms nvdiffrec+FlexiCubes in terms of PSNR while performing on par on SSIM and LPIPS. However, we note that nvdiffrec with both DMTet and FlexiCubes requires visibility masks, while our method does not. Further, our method obtains performance competitive with a state-of-the-art surface-based, non-mesh rendering method NeuS2 [Wang et al. 2021], achieving a PSNR within 0.36 dB. Zip-NeRF, which is not regularized to obtain smooth surfaces, outperforms all other methods. We further compare against Volumetric Surfaces, which similarly trains a collection of semi-transparent shells, but initializes from the NeuS2 output and does not further deform mesh geometry. For evaluation, we extract their 0-level set surface and assess performance using the same novel view synthesis metrics. We outperform their method by approximately 2 dB.

Method	PSNR ↑	SSIM ↑	LPIPS ↓
Ours (All Layers)	29.36	0.939	0.077
Ours (Zero Level Set)	29.37	0.939	0.077
nvdiffrec (DMTet)	28.80	0.938	0.078
nvdiffrec (Flexicubes)	29.22	0.940	0.076
Neus2	<u>29.72</u>	<u>0.943</u>	<u>0.068</u>
Zip-NeRF	**33.10**	**0.971**	**0.031**
Volumetric Surfaces	27.33	0.919	0.110

Table 1. Results on the NeRF Synthetic dataset. Our method outperforms differentiable mesh rendering methods (top) and approaches the performance of representative volumetric, surface-based, and primitive based methods (bottom).

4.3 Mip-NeRF 360 Reconstruction

Next, we compare our method to several non-mesh-based differentiable rendering approaches on the Mip-NeRF 360 dataset. Consistent with observations made in related work [Reiser et al. 2024], our

best attempts at reconstructing this dataset with prior mesh-based differentiable rendering method nvdiffrec [Munkberg et al. 2022b] with both a sphere-based initialization and a FlexiCubes [Shen et al. 2023] scene representation failed to produce a mesh that resembled the scene.

We show qualitiative results in Figure 8 and quantitative results in Table 2. Qualitatively, our method obtains crisp and high-quality renderings that capture much of even the high-frequency detail of the scene, such as the spokes of the bike in BICYCLE, the leaves of trees and bushes in the background of GARDEN, or the fine detail in the leaves of BONSAI. However, our method does not capture quite as many details as Zip-NeRF and displays some high-frequency artifacts around some edges, which are visible with BONSAI. Consistent with this qualitative impression, our quantitative results achieve 24.23 dB, lagging behind Gaussian Splatting by 3 dB, which, in turn, lags Zip-NeRF by about 1.3 dB. Our method is the first mesh-based differentiable rendering method that approaches these state-of-the-art methods, validating that mesh-based differentiable rendering is in principle capable of competitive performance and suggesting that custom differentiable renderers may not be critical for competitive real-world differentiable rendering.

Method	PSNR ↑	SSIM ↑	LPIPS ↓
Ours (All Layers)	24.34	0.686	0.367
Ours (Zero Level Set)	24.30	0.684	0.369
Zip-NeRF	**28.54**	**0.828**	**0.189**
3D Gaussian Splatting	<u>27.21</u>	<u>0.815</u>	<u>0.214</u>

Table 2. Results on the Mip-NeRF 360 dataset. Our method is the only differentiable mesh rendering method that is capable of reconstructing real-world scenes (top). We compare its performance against gold-standard volumetric and primitive based methods (bottom).

4.4 Analysis

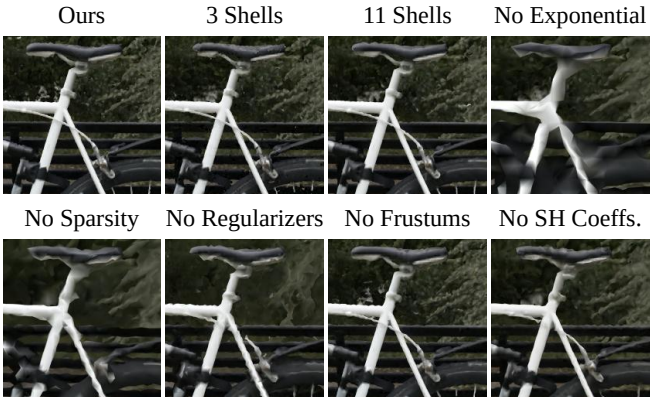


Fig. 6. We ablate our method's hyperparameter and design choices. See section 4.4 for an explanation of each ablation.

To validate our method's design, we ablate several key components on Mip-NeRF 360's GARDEN scene, quantitatively in Table 3 and qualitatively in Figure 6. In the "no exponential" ablation, we distribute our mesh vertices evenly in space rather than using the exponential vertex positioning. In the "no sparsity" ablation, we maintain a constant memory budget and use a lower-resolution dense 3D grid instead of the sparse grid discussed in section 3.3. In the "no regularizers" ablation, we remove the regularizers discussed in section 3.5. This ablation demonstrates that our regularizers are essential for well-behaved optimization that yields high-quality results. In the "no frustums" ablation, we use a cubic grid for both the foreground and the background rather than using truncated frustums for the background. In the "no SH coeffs." ablation, we disable spherical harmonics and use view-independent colors. These ablations all yield lower quality for the same memory budget. Finally, we ablate the number of shells used for training in the "3 shells" and "11 shells" ablations (we generally use 5 shells).

Method	PSNR ↑	SSIM ↑	LPIPS ↓
Ours	**21.87**	**0.520**	**0.418**
3 Shells	21.00	0.445	0.473
11 Shells	<u>21.83</u>	<u>0.512</u>	<u>0.438</u>
No Exponential	19.84	0.376	0.622
No Sparsity	20.53	0.405	0.616
No Regularizers	20.55	0.410	0.569
No Frustums	21.56	0.489	0.471
No SH Coeffs.	20.71	0.430	0.551

Table 3. We validate our design decisions by ablating several key components of our method. See section 4.4 for descriptions of each ablation.

4.5 Limitations

While our method is able to reconstruct both real-world and object-centric scenes, it has limitations that we hope to address in future work. First, our method's number of triangles is larger than the number of primitives in existing primitive-based approaches. This could be addressed by using adaptive iso-surface extraction methods (e.g., based on octrees). Second, similar to primitive-based Gaussian Splatting methods, our method's optimization dynamics can be less well-behaved than those of NeRF-like methods. In particular, our method will sometimes create thin "floaters" which we show in Figure 7. Since NeRF was introduced, NeRF-like methods have overcome this challenge with sophisticated regularization techniques. Similar regularization techniques compatible with our representation are left to future work.

5 DISCUSSION AND CONCLUSION

We have introduced a novel differentiable mesh rendering framework that leverages a non-differentiable triangle rasterizer to optimize a 3D mesh representation of a scene. Our work departs from current mainstream approaches to differentiable rendering based on volumetric radiance field representations, offering a fresh theoretical perspective as well as novel algorithmic tools. Our method demonstrates that there exists a path towards matching today's impressive differentiable rendering results using only pieces of the conventional computer graphics rendering stack.

Fig. 7. Limitations. Like Gaussian Splatting, our method struggles to accurately reconstruct low camera angles and fine details in regions that are sparsely covered by the training views. Gaussian Splatting tends to blur these regions; our method creates undesirable high-frequency artifacts.

REFERENCES

Sai Praveen Bangaru, Lifan Wu, Tzu-Mao Li, Jacob Munkberg, Gilbert Bernstein, Jonathan Ragan-Kelley, Fredo Durand, Aaron Lefohn, and Yong He. 2023. Slang. d: Fast, modular and differentiable shader programming. *ACM Transactions on Graphics (TOG)* 42, 6 (2023), 1–28.

Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. 2022. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields. *CVPR* (2022).

Alexandre Binninger, Ruben Wiersma, Philipp Herholz, and Olga Sorkine-Hornung. 2025. TetWeave: Isosurface Extraction using On-The-Fly Delaunay Tetrahedral Grids for Gradient-Based Mesh Optimization. *ACM Transactions on Graphics (TOG)* (2025).

Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. 2023. MobileNeRF: Exploiting the Polygon Rasterization Pipeline for Efficient Neural Field Rendering on Mobile Architectures. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Evgeni V. Chernyaev. 1995. Marching Cubes 33: Construction of Topologically Correct Isosurfaces. https://api.semanticscholar.org/CorpusID:18816939

Jaehoon Choi, Rajvi Shah, Qinbo Li, Yipeng Wang, Ayush Saraf, Changil Kim, Jia-Bin Huang Dinesh Manocha, Suhib Alsisan, and Johannes Kopf. 2024. LTM: Lightweight Textured Mesh Extraction and Refinement of Large Unbounded Scenes for Efficient Storage and Real-time Rendering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Thomas Deliot, Eric Heitz, and Laurent Belcour. 2024. Transforming a Non-Differentiable Rasterizer into a Differentiable One with Stochastic Gradient Estimation. *arXiv preprint arXiv:2404.09758* (2024).

Akio Doi and Akio Koide. 1991. An Efficient Method of Triangulating Equi-Valued Surfaces by Using Tetrahedral Cells. *IEICE TRANSACTIONS on Information* E74-D, 1 (January 1991), 214–224.

Stefano Esposito, Anpei Chen, Christian Reiser, Samuel Rota Bulò, Lorenzo Porzi, Katja Schwarz, Christian Richardt, Michael Zollhöfer, Peter Kontschieder, and Andreas Geiger. 2024. Volumetric Surfaces: Representing Fuzzy Geometries with Layered Meshes. *arXiv preprint arXiv:2409.02482* (2024).

Fridovich-Keil and Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. 2022. Plenoxels: Radiance Fields without Neural Networks. In *CVPR*.

Jun Gao, Wenzheng Chen, Tommy Xiang, Clement Fuji Tsang, Alec Jacobson, Morgan McGuire, and Sanja Fidler. 2020. Learning Deformable Tetrahedral Meshes for 3D Reconstruction. In *Advances In Neural Information Processing Systems*.

Purvi Goel, Loudon Cohen, James Guesman, Vikas Thamizharasan, James Tompkin, and Daniel Ritchie. 2020. Shape from Tracing: Towards Reconstructing 3D Object Geometry and SVBRDF Material from Images via Differentiable Path Tracing . In *2020 International Conference on 3D Vision (3DV)*. IEEE Computer Society, Los Alamitos, CA, USA, 1186–1195. https://doi.org/10.1109/3DV50981.2020.00129

Antoine Guédon and Vincent Lepetit. 2024. SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2024).

Benoît Guillard, Edoardo Remelli, Artem Lukoianov, Pierre Yvernay, Stephan R. Richter, Timur Bagautdinov, Pierre Baque, and Pascal Fua. 2024. DeepMesh: Differentiable Iso-Surface Extraction. *IEEE Trans. Pattern Anal. Mach. Intell.* 46, 11 (Nov. 2024), 7072–7087. https://doi.org/10.1109/TPAMI.2024.3392291

Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2024. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. In *SIGGRAPH 2024 Conference Papers*. Association for Computing Machinery. https://doi.org/10.1145/3641519.3657428

Tao Ju, Frank Losasso, Scott Schaefer, and Joe Warren. 2002. Dual contouring of hermite data. *ACM Trans. Graph.* 21, 3 (July 2002), 339–346. https://doi.org/10.1145/566654.566586

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics (TOG)* 42, 4 (July 2023). https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/

Samuli Laine, Janne Hellsten, Tero Karras, Yeongho Seol, Jaakko Lehtinen, and Timo Aila. 2020a. Modular Primitives for High-Performance Differentiable Rendering. *ACM Transactions on Graphics (TOG)* 39, 6 (2020).

Samuli Laine, Janne Hellsten, Tero Karras, Yeongho Seol, Jaakko Lehtinen, and Timo Aila. 2020b. Modular Primitives for High-Performance Differentiable Rendering. *ACM Transactions on Graphics* 39, 6 (2020).

Yiyi Liao, Simon Donné, and Andreas Geiger. 2018. Deep Marching Cubes: Learning Explicit Surface Representations. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.

Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. 2019. Soft Rasterizer: A Differentiable Renderer for Image-based 3D Reasoning. *Proceedings of the International Conference on Computer Vision (ICCV)* (Oct 2019).

Adriano Lopes and Ken Brodlie. 2003. Improving the Robustness and Accuracy of the Marching Cubes Algorithm for Isosurfacing. *IEEE Transactions on Visualization and Computer Graphics* 9, 1 (Jan. 2003), 16–29. https://doi.org/10.1109/TVCG.2003.1175094

William E. Lorensen and Harvey E. Cline. 1987. Marching cubes: A high resolution 3D surface construction algorithm. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*. Association for Computing Machinery, New York, NY, USA, 163–169. https://doi.org/10.1145/37401.37422

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Transactions on Graphics (TOG)* 41, 4, Article 102 (July 2022), 15 pages. https://doi.org/10.1145/3528223.3530127

Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. 2022a. Extracting Triangular 3D Models, Materials, and Lighting From Images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 8280–8290.

Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. 2022b. Extracting Triangular 3D Models, Materials, and Lighting From Images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 8280–8290.

Thomas Neff, Pascal Stadlbauer, Mathias Parger, Andreas Kurz, Joerg H. Mueller, Chakravarty R. Alla Chaitanya, Anton S. Kaplanyan, and Markus Steinberger. 2021. DONeRF: Towards Real-Time Rendering of Compact Neural Radiance Fields using Depth Oracle Networks. *Computer Graphics Forum* 40, 4 (2021). https://doi.org/10.1111/cgf.14340

Baptiste Nicolet, Alec Jacobson, and Wenzel Jakob. 2021. Large Steps in Inverse Rendering of Geometry. *ACM Transactions on Graphics (Proceedings of SIGGRAPH Asia)* 40, 6 (Dec. 2021). https://doi.org/10.1145/3478513.3480501

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 [cs.LG] https://arxiv.org/abs/1912.01703

Christian Reiser, Stephan Garbin, Pratul P. Srinivasan, Dor Verbin, Richard Szeliski, Ben Mildenhall, Jonathan T. Barron, Peter Hedman, and Andreas Geiger. 2024. Binary Opacity Grids: Capturing Fine Geometric Detail for Mesh-Based View Synthesis. *ACM Transactions on Graphics (TOG)* (2024).

Christian Reiser, Rick Szeliski, Dor Verbin, Pratul Srinivasan, Ben Mildenhall, Andreas Geiger, Jon Barron, and Peter Hedman. 2023. MERF: Memory-Efficient Radiance Fields for Real-time View Synthesis in Unbounded Scenes. *ACM Trans. Graph.* 42, 4, Article 89 (July 2023), 12 pages. https://doi.org/10.1145/3592426

Edoardo Remelli, Artem Lukoianov, Stephan Richter, Benoit Guillard, Timur Bagautdinov, Pierre Baque, and Pascal Fua. 2020. MeshSDF: Differentiable Iso-Surface Extraction. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 22468–22478. https://proceedings.neurips.cc/paper/2020/file/fe40fb944ee700392ed51bfe84dd4e3d-Paper.pdf

Gopal Sharma, Daniel Rebain, Kwang Moo Yi, and Andrea Tagliasacchi. 2024. Volumetric rendering with baked quadrature fields. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 275–292.

Tianchang Shen, Jun Gao, Kangxue Yin, Ming-Yu Liu, and Sanja Fidler. 2021. Deep Marching Tetrahedra: a Hybrid Representation for High-Resolution 3D Shape Synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tianchang Shen, Jacob Munkberg, Jon Hasselgren, Kangxue Yin, Zian Wang, Wenzheng Chen, Zan Gojcic, Sanja Fidler, Nicholas Sharp, and Jun Gao. 2023. Flexible Isosurface Extraction for Gradient-Based Mesh Optimization. *ACM Transactions on Graphics (TOG)* 42, 4, Article 37 (jul 2023), 16 pages. https://doi.org/10.1145/3592430

Cheng Sun, Min Sun, and Hwann-Tzong Chen. 2022. Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Ziyu Wan, Christian Richardt, Aljaž Božič, Chao Li, Vijay Rengarajan, Seonghyeon Nam, Xiaoyu Xiang, Tuotuo Li, Bo Zhu, Rakesh Ranjan, et al. 2023. Learning neural duplex radiance fields for real-time view synthesis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 8307–8316.

Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. 2021. NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction. *arXiv preprint arXiv:2106.10689* (2021).

Yiming Wang, Qin Han, Marc Habermann, Kostas Daniilidis, Christian Theobalt, and Lingjie Liu. 2023a. NeuS2: Fast Learning of Neural Implicit Surfaces for Multi-view Reconstruction. In *Proceedings of the International Conference on Computer Vision (ICCV)*.

Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612. https://doi.org/10.1109/TIP.2003.819861

Zian Wang, Tianchang Shen, Merlin Nimier-David, Nicholas Sharp, Jun Gao, Alexander Keller, Sanja Fidler, Thomas Müller, and Zan Gojcic. 2023b. Adaptive shells for efficient neural radiance field rendering. *ACM Transactions on Graphics (TOG)* (2023).

Xinyue Wei, Fanbo Xiang, Sai Bi, Anpei Chen, Kalyan Sunkavalli, Zexiang Xu, and Hao Su. 2025a. Neumanifold: Neural watertight manifold reconstruction with efficient and high-quality rendering support. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 731–741.

Xinyue Wei, Fanbo Xiang, Sai Bi, Anpei Chen, Kalyan Sunkavalli, Zexiang Xu, and Hao Su. 2025b. Neumanifold: Neural watertight manifold reconstruction with efficient and high-quality rendering support. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 731–741.

Lior Yariv, Jiatao Gu, Yoni Kasten, and Yaron Lipman. 2021. Volume rendering of neural implicit surfaces. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Lior Yariv, Peter Hedman, Christian Reiser, Dor Verbin, Pratul P. Srinivasan, Richard Szeliski, Jonathan T. Barron, and Ben Mildenhall. 2023. BakedSDF: Meshing Neural SDFs for Real-Time View Synthesis. *ACM Transactions on Graphics (TOG)* (2023).

Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. 2021. PlenOctrees for Real-time Rendering of Neural Radiance Fields. In *Proceedings of the International Conference on Computer Vision (ICCV)*.

Zehao Yu, Torsten Sattler, and Andreas Geiger. 2024. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–13.

Kai Zhang, Gernot Riegler, Noah Snavely, and Vladlen Koltun. 2020. NeRF++: Analyzing and Improving Neural Radiance Fields. *arXiv:2010.07492* (2020).

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 586–595. https://doi.org/10.1109/CVPR.2018.00068

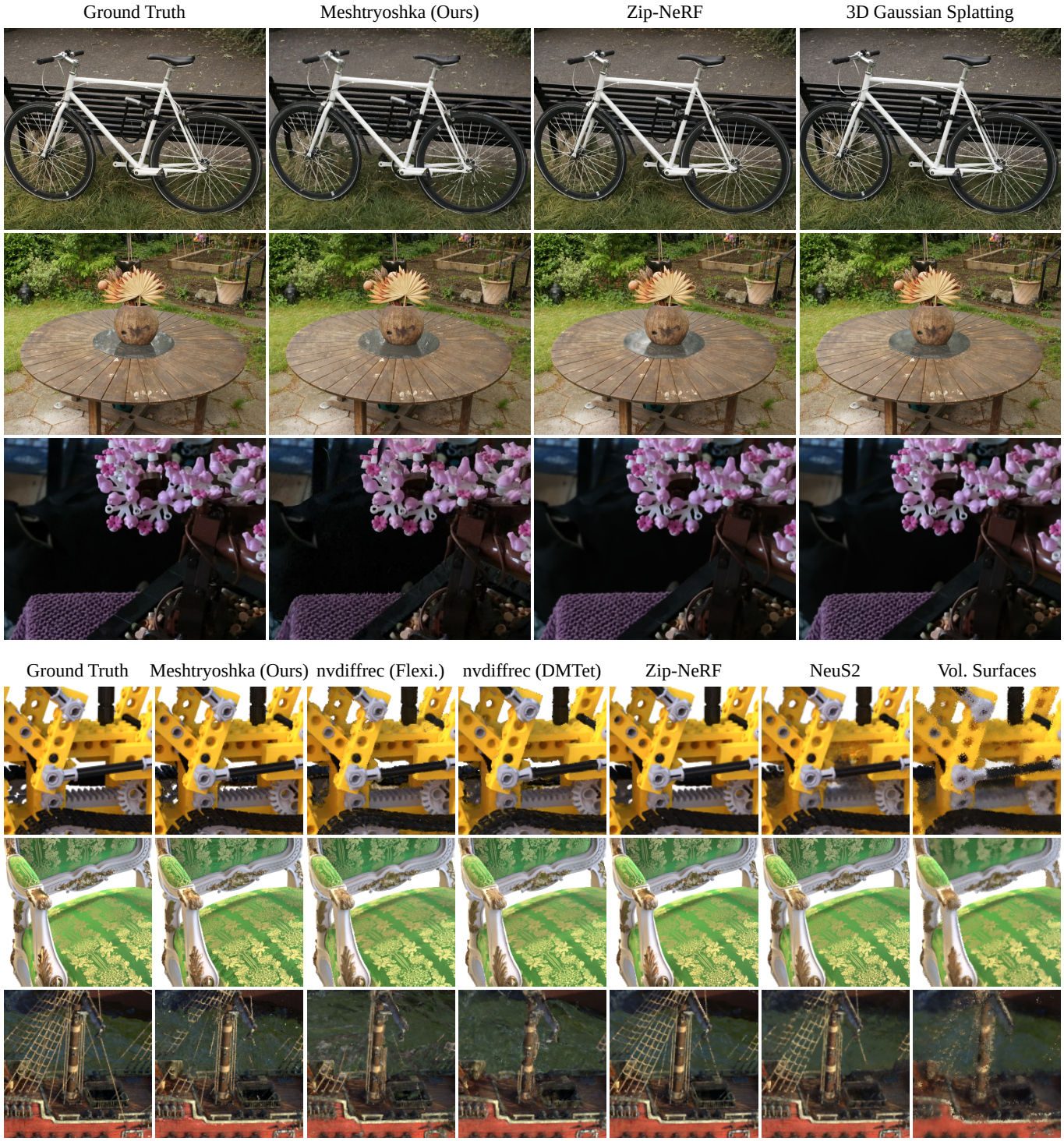


Fig. 8. Our method's reconstruction quality on object-centric scenes matches nvdiffrec, an existing mesh-based differentiable rendering framework, and approaches that of non-mesh-based frameworks. Our method is the only mesh-based differentiable rendering method that is capable of reconstructing real-world scenes, although the visual fidelity it produces is somewhat below that of non-mesh-based methods.